Code:-

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import PolynomialFeatures, StandardScaler
from sklearn.linear_model import LinearRegression
from sklearn.pipeline import Pipeline
from sklearn.metrics import (
    mean_squared_error,
    mean_absolute_error,
    r2_score
)

# Load dataset
data = pd.read_csv("/home/avcoe/Housing.csv")

print("Dataset loaded successfully.")

# Display dataset information
print("\nDataset shape:")
print(data.shape)

print("\nFirst 5 rows:")
print(data.head())

print("\nColumn names:")
print(data.columns.tolist())

print("\nData types:")
print(data.dtypes)

print("\nMissing values in original dataset:")
print(data.isnull().sum())

# Define target column
TARGET_COLUMN = "price"

if TARGET_COLUMN not in data.columns:
    raise ValueError(
        f"Target column '{TARGET_COLUMN}' not found.\n"
        f"Available columns are:\n"
        f"{data.columns.tolist()}"
    )
```

```
# Separate input features and target
X = data.drop(columns=[TARGET_COLUMN])
y = data[TARGET_COLUMN]

print("\nInput shape:")
print(X.shape)

print("\nTarget shape:")
print(y.shape)

# Identify numerical and categorical features
numeric_columns = X.select_dtypes(include=["number"]).columns
categorical_columns = X.select_dtypes(
   include=["object", "category"]
).columns

print("\nNumerical columns:")
print(numeric_columns.tolist())

print("\nCategorical columns:")
print(categorical_columns.tolist())

# Handle missing numerical values
X[numeric_columns] = X[numeric_columns].fillna(
   X[numeric_columns].median()
)

# Handle missing categorical values
for column in categorical_columns:
   if X[column].isnull().any():
      X[column] = X[column].fillna(
         X[column].mode()[0]
      )

# Convert categorical columns to numeric
X = pd.get_dummies(
   X,
   columns=categorical_columns,
   drop_first=False,
   dtype=int
)

print("\nColumns after categorical encoding:")
print(X.columns.tolist())

print("\nInput data types after encoding:")
print(X.dtypes)
```

```
# Check for missing values
print("\nMissing values after preprocessing:")
print(X.isnull().sum())

print(
   "\nTotal missing input values:",
   X.isnull().sum().sum()
)

print(
   "Missing target values:",
   y.isnull().sum()
)

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.20,  random_state=42)

print("\nTraining samples:")
print(len(X_train))

print("\nTesting samples:")
print(len(X_test))

print(  "\nNumber of input features after encoding:", X.shape[1])

# Linear Regression
linear_model = LinearRegression()
linear_model.fit(X_train, y_train)
linear_predictions = linear_model.predict(X_test)

# Linear Regression metrics
linear_mse = mean_squared_error( y_test, linear_predictions)
linear_mae = mean_absolute_error(y_test, linear_predictions)
linear_r2 = r2_score( y_test,linear_predictions)

print("\nLINEAR REGRESSION")
print(  f"Mean Squared Error (MSE) : {linear_mse:.6f}")
print(    f"Mean Absolute Error (MAE): {linear_mae:.6f}")
print(    f"R² Score  : {linear_r2:.6f}")
```

```
# Polynomial Regression
polynomial_model = Pipeline(
   [
      (
         "polynomial_features",
         PolynomialFeatures(
            degree=2,
            include_bias=False
         )
      ),
      (
         "scaler",
         StandardScaler()
      ),
      (
         "linear_regression",
         LinearRegression()
      )
   ]
)

# Train polynomial model
polynomial_model.fit(X_train, y_train)

# Polynomial prediction
polynomial_predictions = polynomial_model.predict(X_test)

# Polynomial Regression metrics
polynomial_mse = mean_squared_error(y_test, polynomial_predictions)
polynomial_mae = mean_absolute_error(y_test, polynomial_predictions)

polynomial_r2 = r2_score(y_test, polynomial_predictions)

print("\nPOLYNOMIAL REGRESSION")
print(f"Mean Squared Error (MSE) : {polynomial_mse:.6f}")
print(  f"Mean Absolute Error (MAE): {polynomial_mae:.6f}")
print(  f"R² Score   : {polynomial_r2:.6f}")
```

```
# Display sample predictions
results = pd.DataFrame(
  {
    "Actual Price":
      y_test.values[:10],

    "Linear Prediction":
      linear_predictions[:10],

    "Polynomial Prediction":
      polynomial_predictions[:10]
  }
)

print("\nSAMPLE PREDICTIONS")
print(results.to_string(index=False))

# Model comparison
comparison = pd.DataFrame(
  {
    "Model": [
      "Linear Regression",
      "Polynomial Regression"
    ],

    "MSE": [
      linear_mse,
      polynomial_mse
    ],

    "MAE": [
      linear_mae,
      polynomial_mae
    ],

    "R2": [
      linear_r2,
      polynomial_r2
    ]
  }
)

print("\nMODEL COMPARISON")
print(comparison.to_string(index=False))
```

Output:-

Dataset loaded successfully.

Dataset shape:
(545, 13)

First 5 rows:

	price	area	bedrooms	...	parking	prefarea	furnishingstatus
0	13300000	7420	4	...	2	yes	furnished
1	12250000	8960	4	...	3	no	furnished
2	12250000	9960	3	...	2	yes	semi-furnished
3	12215000	7500	4	...	3	yes	furnished
4	11410000	7420	4	...	2	no	furnished

[5 rows x 13 columns]

Column names:
['price', 'area', 'bedrooms', 'bathrooms', 'stories', 'mainroad', 'guestroom', 'basement', 'hotwaterheating', 'airconditioning', 'parking', 'prefarea', 'furnishingstatus']
Data types:
price int64
area int64
bedrooms int64
bathrooms int64
stories int64
mainroad str
guestroom str
basement str
hotwaterheating str
airconditioning str
parking int64
prefarea str
furnishingstatus str
dtype: object

Missing values in original dataset:
price 0
area 0
bedrooms 0
bathrooms 0
stories 0
mainroad 0
guestroom 0
basement 0
hotwaterheating 0
airconditioning 0
parking 0
prefarea 0
furnishingstatus 0
dtype: int64

Input shape:
(545, 12)

Target shape:
(545,)

Numerical columns:
['area', 'bedrooms', 'bathrooms', 'stories', 'parking']

Categorical columns:
['mainroad', 'guestroom', 'basement', 'hotwaterheating', 'airconditioning', 'prefarea', 'furnishingstatus']

Columns after categorical encoding:
['area', 'bedrooms', 'bathrooms', 'stories', 'parking', 'mainroad_no', 'mainroad_yes', 'guestroom_no', 'guestroom_yes', 'basement_no', 'basement_yes', 'hotwaterheating_no', 'hotwaterheating_yes', 'airconditioning_no', 'airconditioning_yes', 'prefarea_no', 'prefarea_yes', 'furnishingstatus_furnished', 'furnishingstatus_semi-furnished', 'furnishingstatus_unfurnished']

Input data types after encoding:
area int64
bedrooms int64
bathrooms int64
stories int64
parking int64
mainroad_no int64
mainroad_yes int64
guestroom_no int64
guestroom_yes int64
basement_no int64
basement_yes int64
hotwaterheating_no int64
hotwaterheating_yes int64
airconditioning_no int64
airconditioning_yes int64
prefarea_no int64
prefarea_yes int64
furnishingstatus_furnished int64
furnishingstatus_semi-furnished int64
furnishingstatus_unfurnished int64
dtype: object

Missing values after preprocessing:
area 0
bedrooms 0
bathrooms 0
stories 0
parking 0
mainroad_no 0
mainroad_yes 0
guestroom_no 0
guestroom_yes 0
basement_no 0
basement_yes 0
hotwaterheating_no 0
hotwaterheating_yes 0
airconditioning_no 0
airconditioning_yes 0
prefarea_no 0
prefarea_yes 0
furnishingstatus_furnished 0
furnishingstatus_semi-furnished 0
furnishingstatus_unfurnished 0
dtype: int64

Total missing input values: 0
Missing target values: 0
/tmp/ipykernel_2478/4182528163.py:3: Pandas4Warning: For backward compatibility, 'str' dtypes are included by select_dtypes when 'object' dtype is specified. This behavior is deprecated and will be removed in a future version. Explicitly pass 'str' to `include` to select them, or to `exclude` to remove them and silence this warning.
See https://pandas.pydata.org/docs/user_guide/migration-3-strings.html#string-migration-select-dtypes for details on how to write code that works with pandas 2 and 3.
  categorical_columns = X.select_dtypes(include=["object","category"]).columns

Training samples:
436
Testing samples:
109

Number of input features after encoding: 20

LINEAR REGRESSION
Mean Squared Error (MSE) : 1754318687330.660889
Mean Absolute Error (MAE): 970043.403920
R^2 Score : 0.652924

POLYNOMIAL REGRESSION
Mean Squared Error (MSE) : 1907119591880.386963
Mean Absolute Error (MAE): 1038245.065965
R^2 Score : 0.622694

SAMPLE PREDICTIONS

Actual Price	Linear Prediction	Polynomial Prediction
4060000	5.164654e+06	4.848019e+06
6650000	7.224722e+06	7.518483e+06
3710000	3.109863e+06	3.904826e+06
6440000	4.612075e+06	4.459131e+06
2800000	3.294646e+06	3.589901e+06
4900000	3.532275e+06	3.432534e+06
5250000	5.611775e+06	5.446544e+06
4543000	6.368146e+06	5.394060e+06
2450000	2.722857e+06	3.589884e+06
3353000	2.629406e+06	3.139914e+06

MODEL COMPARISON

Model	MSE	MAE	R2
Linear Regression	1.754319e+12	9.700434e+05	0.652924
Polynomial Regression	1.907120e+12	1.038245e+06	0.622694